federated-learning research security portfolio

Building a Research Portfolio in Federated Learning Security

Lessons from building a comprehensive federated learning security research suite — covering attacks, defenses, and privacy techniques across 165,000+ lines of research-grade code.

February 10, 2026 10 min read Azka

Over the past year, I developed a comprehensive research suite focused entirely on federated learning security. This wasn't a bootcamp or a challenge—it was a deliberate deep-dive into understanding how distributed ML systems fail and how to defend them.

Here's what I learned from building this body of work.

The Motivation

After 3+ years in Indonesian banking fraud detection, I had production ML experience. But federated learning security remained theoretical—something I read about in papers but never implemented.

The gap bothered me. Papers describe attacks and defenses in abstract terms: "Krum achieves Byzantine resilience by selecting the update with minimum distance to its neighbors." But what does that actually look like in code? How does it fail? When does it work?

I wanted to bridge theory and practice—not just understand the concepts, but build working implementations I could examine, break, and improve.

Research Structure

I organized the research suite into five thematic categories covering attacks, defenses, and privacy techniques. The work comprises **165,000+ lines of code** across **101 test files**, with security validated through a comprehensive **STRIDE audit**.

Foundation: Fraud Detection & FL Basics

Foundation: Fraud Detection & FL Basics

The starting point was implementing standard federated learning on realistic fraud detection data. This established baselines for accuracy, communication overhead, and client heterogeneity.

Key projects included:

- Baseline FL with FedAvg on credit card fraud data
- Client heterogeneity analysis (different banks, different distributions)
- Communication efficiency benchmarks

Attacks: Understanding the Threat Landscape

Before defending, I needed to understand attacks. I implemented the major attack categories:

- **Data poisoning**: Label flipping, dirty label attacks
- **Model poisoning**: Sign flipping, scaling attacks, Gaussian noise
- **Backdoors**: Trigger-based, semantic, and adaptive backdoors
- **Gradient leakage**: Reconstruction attacks on batch data

Each attack was tested against standard FedAvg to establish vulnerability baselines.

Defenses: Algorithmic Approaches

The defense implementations focused on robust aggregation:

- Krum and Multi-Krum variants
- Trimmed mean and coordinate-wise median
- Clustering-based outlier detection
- Reputation-weighted aggregation

This is where I discovered that most algorithmic defenses share a critical weakness: they're brittle to adaptive attacks.

Privacy: Differential Privacy & Secure Aggregation

Beyond robustness, FL requires privacy:

- Differential privacy (DP-SGD, local DP)
- Secure aggregation protocols
- Homomorphic encryption experiments
- Privacy-utility trade-off analysis

Cryptography: The SignGuard Direction

The final research arc explored cryptographic defenses:

- ECDSA signature verification for update integrity
- Anomaly detection for update quality validation
- Threshold signatures for multi-party verification

This became the foundation for SignGuard—my most significant contribution from this research.

Technical Insights

Byzantine Robustness Is Fragile

Krum and its variants work well in controlled settings. But they make assumptions that don't hold in production:

Assumption	Reality
Malicious clients < 50%	No guarantee in open FL
Attacks are random	Adaptive attacks target defenses
Static threat model	Attackers evolve

Multi-Krum with reputation weighting performed best in my tests, but still failed against sophisticated adaptive attacks.

Algorithmic vs. Cryptographic Defenses

Most FL security research focuses on algorithmic defenses—statistical methods to detect outliers. But I found cryptographic approaches offer stronger guarantees:

Algorithmic defenses (Krum, clustering):

- Probabilistic detection
- Adaptive attacks bypass them
- No accountability

Cryptographic defenses (signatures, secure aggregation):

- Deterministic verification
- Resistant to manipulation
- Full audit trail

This insight shifted my research direction toward SignGuard.

Real Data Exposes Hidden Assumptions

Synthetic benchmarks (FEMNIST, Shakespeare) are useful for prototyping. But real financial transaction data revealed issues I never saw in simulations:

- **Extreme imbalance**: 0.1% fraud rate means 1000:1 class imbalance
- **Temporal drift**: Fraud patterns shift weekly, not just monthly
- **Geographic heterogeneity**: Jakarta fraud ≠ Bali fraud ≠ Papua fraud
- **Missing data**: Real transactions have gaps, errors, delays

Models trained on clean synthetic data broke on real data.

Research Methodology Lessons

1. Start with Reference Implementations

Don't build from scratch. Clone Flower, FedML, or PySyft. Modify them. Building infrastructure from scratch wastes time you could spend on research questions.

2. Establish Baselines First

Before testing defenses, measure attack success rates on undefended systems. Without baselines, you can't quantify improvement.

3. Document Decisions Immediately

Around mid-way through the research, I couldn't remember why I chose certain parameters in earlier implementations. I started keeping decision logs: why FedAvg over

FedProx, why Krum failed in this specific test, etc.

These logs became more valuable than the code itself.

4. Test Against Adaptive Adversaries

Static attack tests are insufficient. Implement attacks that know your defense and try to bypass it. This is where most algorithmic defenses fail.

5. Run Real Benchmarks from Day One

I initially relied on theoretical projections for attack success rates. When I finally implemented empirical benchmarks, the results differed significantly from expectations. Real benchmarks expose hidden assumptions that theoretical analysis misses.

Career Impact

This portfolio transformed my technical trajectory:

Before: Fraud detection practitioner using ML as a tool **After**: ML security researcher contributing to the field

Concrete outcomes:

- Conference presentations on FL security
- Collaborations with academic researchers
- SignGuard as a novel contribution with measurable results
- Deeper understanding of my fraud detection work through the security lens

What I'd Do Differently

Run real benchmarks from day one: I initially relied on theoretical projections. When I finally implemented empirical benchmarks, results differed significantly from expectations. Real benchmarks expose hidden assumptions that theoretical analysis misses.

Start with defense work earlier: I should have started with SignGuard earlier and built the attack implementations as motivation for it. Working backwards from the defense to understand what attacks it needs to stop would have been more efficient.

Focus over breadth: A focused deep dive on fewer, deeper implementations would have been more impactful than covering many techniques. Quality over quantity.

Open-source from the start: I kept code private initially. Sharing earlier would have attracted collaborators and accelerated feedback cycles.

Resources for Similar Research

Foundational Papers:

- McMahan et al. “Communication-Efficient Learning of Deep Networks from Decentralized Data” (2017) - FedAvg
- Blanchard et al. “Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent” (2017) - Krum
- Bonawitz et al. “Practical Secure Aggregation for Privacy-Preserving Machine Learning” (2017)

Frameworks:

- Flower - Best documentation, active community
- FedML - More research-oriented
- PySyft - Privacy-focused

Communities:

- Flower Discord - Practical implementation help
- Federated Learning Slack - Academic discussions
- r/MachineLearning - Broader ML community

Conclusion

Building this research portfolio was the most valuable technical investment I’ve made. It transformed abstract security concepts into concrete implementations—and gave me a toolkit I use daily in fraud detection work.

The key insight: **research portfolios aren’t about quantity. They’re about depth, coherence, and contribution.**

If you’re considering similar deep-dive research: start with clear questions, build on existing frameworks, and document everything. The portfolio will emerge naturally from genuine curiosity.

This post summarizes insights from my FL Security Research Suite. The code is open-source with 165,000+ lines across 30 implementations, all documented with decision logs and validated through a STRIDE security audit.

Related Posts

← Back to all posts

Azka Al Azkiyai
AI Security Researcher

© 2026 Azka Al Azkiyai